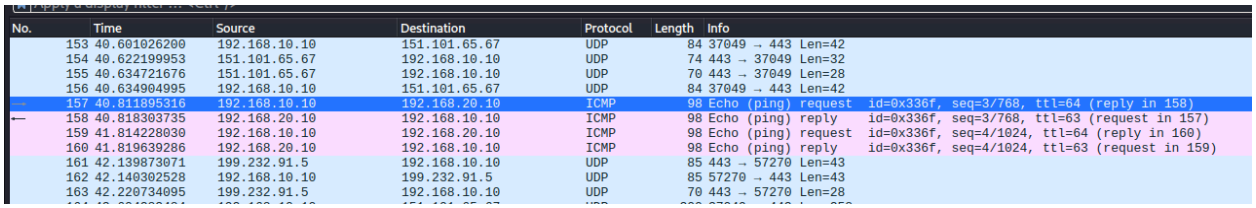
Home Lab Project Week 11, Day 1
Task:
Re-run selected reconnaissance and port-scanning activities from the previous testing phases and capture the traffic with Wireshark. Compare the results before and after firewall implementation.

Phase 1: Preparation & Wireshark Capture

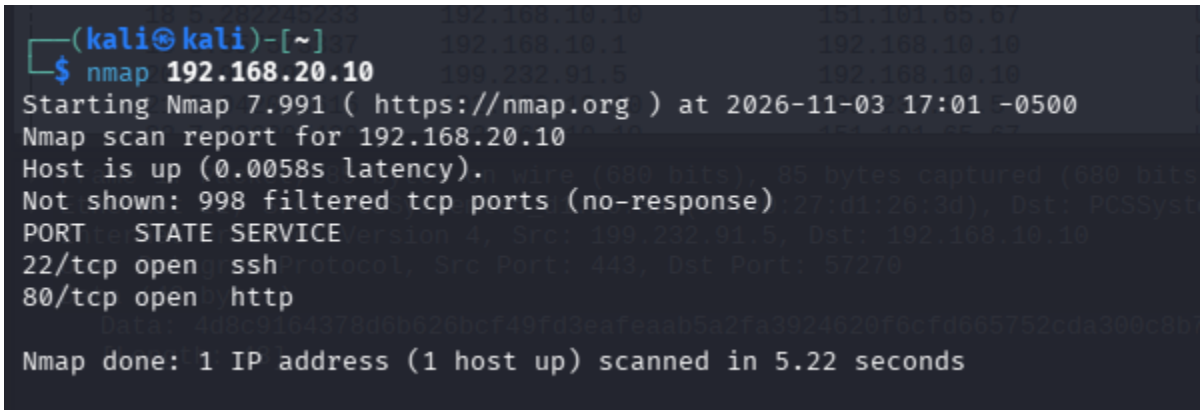
To get clean comparison data, you will run two distinct testing phases.

Test 1: Pre-Firewall (Baseline)

1. **Disable or bypass** your newly implemented firewall rules temporarily (or target a host/subnet where the rules are not yet applied).
2. Start a **Wireshark capture** on the target interface or a mirrored port.
3. Execute your selected reconnaissance scans (e.g., Nmap, Netcat).
4. Stop the capture and save it as `baseline_recon.pcapng`.

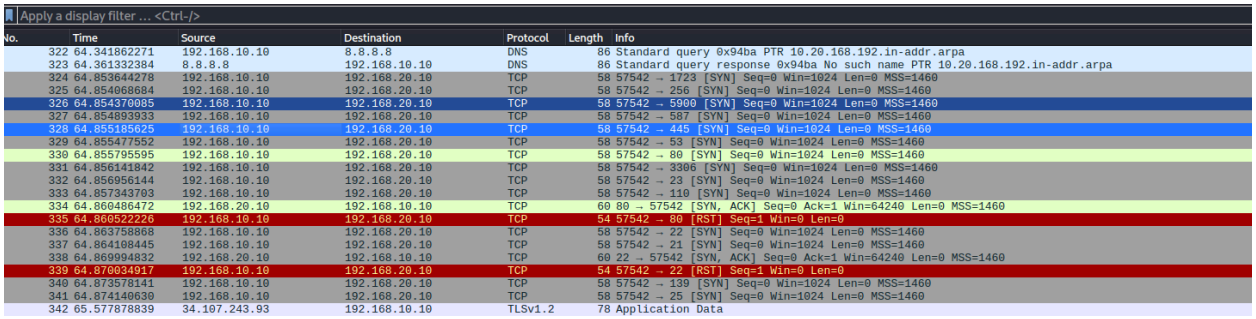


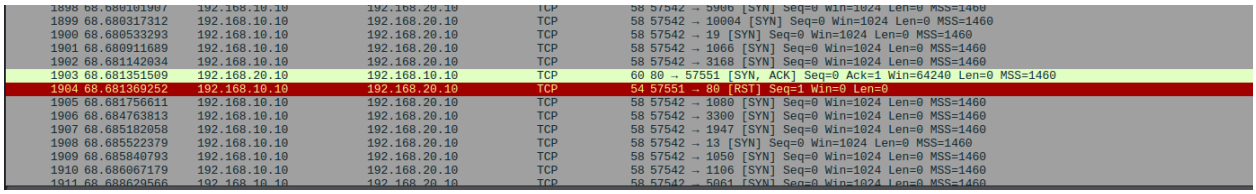
No.	Time	Source	Destination	Protocol	Length	Info
153	40.601026200	192.168.10.10	151.101.65.67	UDP	84	37049 → 443 Len=42
154	40.622199953	151.101.65.67	192.168.10.10	UDP	74	443 → 37049 Len=32
155	40.634721676	151.101.65.67	192.168.10.10	UDP	70	443 → 37049 Len=28
156	40.634904995	192.168.10.10	151.101.65.67	UDP	84	37049 → 443 Len=42
→ 157	40.811895316	192.168.10.10	192.168.20.10	ICMP	98	Echo (ping) request id=0x336f, seq=3/768, ttl=64 (reply in 158)
← 158	40.818303735	192.168.20.10	192.168.10.10	ICMP	98	Echo (ping) reply id=0x336f, seq=3/768, ttl=63 (request in 157)
159	41.814228030	192.168.10.10	192.168.20.10	ICMP	98	Echo (ping) request id=0x336f, seq=4/1024, ttl=64 (reply in 160)
160	41.819639286	192.168.20.10	192.168.10.10	ICMP	98	Echo (ping) reply id=0x336f, seq=4/1024, ttl=63 (request in 159)
161	42.139873071	199.232.91.5	192.168.10.10	UDP	85	443 → 57270 Len=43
162	42.140302528	192.168.10.10	199.232.91.5	UDP	85	57270 → 443 Len=43
163	42.220734095	199.232.91.5	192.168.10.10	UDP	70	443 → 57270 Len=28



```
┌──(kali㉿kali)-[~]
└─$ nmap 192.168.20.10
Starting Nmap 7.991 ( https://nmap.org ) at 2026-11-03 17:01 -0500
Nmap scan report for 192.168.20.10
Host is up (0.0058s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT   STATE SERVICE
22/tcp open  ssh
80/tcp open  http

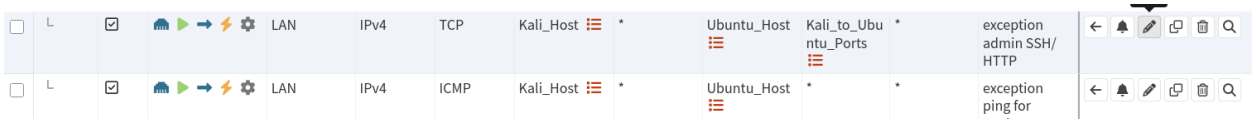
Nmap done: 1 IP address (1 host up) scanned in 5.22 seconds
```





Test 2: Post-Firewall (Hardened)

1. **Enable** your firewall rules (e.g., blocking unauthorized ICMP, dropping stealth scans, or limiting connection rates).
2. Start a **new Wireshark capture** on the same target interface.
3. Execute the exact same reconnaissance scans from the same attacker IP.
4. Stop the capture and save it as `hardened_recon.pcapng`.



Phase 2: Execution Commands (Examples)

If you need to re-run common activities from previous weeks, use these highly visible scanning commands:

- **Ping Sweep (ICMP Recon):**
  bash

```
None

nmap -sn 192.168.1.0/24
```

- Use code with caution.
- **Stealth TCP SYN Scan:**
  bash

```
None

sudo nmap -sS -F 192.168.1.X
```

- Use code with caution.
- **Aggressive Service/OS Detection:**
  bash

```
None

sudo nmap -A -T4 192.168.1.X
```

- Use code with caution.

Phase 3: Side-by-Side Traffic Comparison

Metric / Behavior	Pre-Firewall (Baseline)	Post-Firewall (Hardened)
Target Host Visibility	Hosts respond to ICMP pings and scan requests.	Hosts appear down or completely unresponsive (filtered).

Open Ports Discovered	Lists all active services (e.g., 22, 80, 443).	Shows `0 ports open` or all targeted ports as `filtered`.
Wireshark Traffic Pattern	Clear **SYN** → **SYN-ACK** → **RST** handshake sequences.	**SYN** packets sent by attacker, but **no response** (Timeouts) or **ICMP Unreachable** returned by the firewall.
Scan Completion Time	Fast (seconds to minutes) because the attacker gets immediate responses.	Significantly slower or times out because the attacker is waiting for dropped packets.
Firewall Logs	No alerts or drop actions logged.	Logs populating with `DROP`, `REJECT`, or `BLOCKED` flags from the attacker's IP.

Phase 4: Wireshark Analysis Filters

Use these display filters in Wireshark to quickly pinpoint what changed between your two capture files:

- `ip.src == [Attacker_IP]` → Isolates all traffic coming from your scanning machine.
- `tcp.flags.syn == 1 && tcp.flags.ak == 0` → Displays incoming connection attempts (port scans).
- `icmp` → Checks if ping sweeps are getting through or if the firewall is sending "Destination Unreachable" messages back.